

From Needs to Specifications

Fernando Santana Pacheco

April 2026

Where We Are

Week 4 → Week 5 Transition

You already:

-  Observed the greenhouse
-  Identified problems/opportunities
-  Selected a project direction
-  Submitted a proposal (if not yet, do it!)
- Now we move from:
“Interesting problem” → “Clear design requirements”

Today's Goal

Turn your idea into something actionable

By the end of today:

- You will define **customer needs properly**
- You will organize and prioritize them
- You will translate them into **measurable specifications**

👉 This is what prevents:

- ✗ Jumping too early into building
- ✗ Random features
- ✗ Overcomplicated solutions

The Core Shift

From vague → precise

Instead of:

“Workers need something easier”

We want:

“Workers need to record temperature data in less than 10 seconds without stopping their workflow”

- This is the foundation of good design

Two Approaches to Design

Technology-Centered Design

- "We have this cool LiDAR sensor, what can we do with it?"
- "Let's add AI because it's trendy"
- Starts with technology, looks for problems

User-Centered Design (UCD)

- "Users struggle with navigation in dark warehouses"
- "What technology can help solve this problem?"
- Starts with user needs, selects appropriate technology

Which approach did you use in the initial challenge?

Case Study: Success - iRobot Roomba

User need (not stated explicitly): "I hate vacuuming, but I want clean floors"

NOT: "I need an autonomous mobile robot with SLAM capabilities"

Design decisions driven by UCD:

- Simple one-button operation
- Works unsupervised (user doesn't need to watch)
- Handles "good enough" cleaning (not perfect)
- Fits under furniture (where users can't easily reach)

Result: >50 million units sold ([Fortune](#)), but bankruptcy in 2025 ([IEEE](#))

Case Study: Failure - Google Glass

Technology-centered approach:

"We can put a computer on your face with AR display!"

Ignored user needs:

- Privacy concerns (camera always present)
- Social acceptance (looking weird in public)
- Battery life (insufficient for all-day use)
- Unclear value proposition (what problem does it solve?)

Result: Discontinued for consumer market in 2015

Step 1: What is a Customer Need?

Definition (Ulrich & Eppinger)

A customer need:

- Describes what the user wants
- Is **independent** of any solution
- Is expressed in user language

Bad vs Good Needs

✗ Bad (solution-focused)

- "System should use sensors"
- "Automate the cart"
- "Robot should automate watering"

✓ Good (need-focused)

- "User needs to monitor plant conditions easily"
- "User needs to receive timely alerts"
- "User needs to reduce manual workload"
- Focus on **WHAT**, not **HOW**

Structure of a Good Need

Simple format:

[User/Stakeholder] needs to [verb] [object] [qualifier]

Examples:

- Worker needs to **record data quickly** during tasks
- Manager needs to **access historical data remotely**
- Technician needs to **identify issues early**

Structure of a Good Need (cont.)

Component 1: VERB (action)

- Enable, allow, support, facilitate
- Minimize, reduce, eliminate
- Maximize, increase, enhance
- Provide, deliver, ensure, access
- Maintain, preserve, protect, identify

- Indicate, display, communicate, record

Component 2: OBJECT (what is being acted upon)

- A function, feature, or attribute
- Should be specific

Component 3: QUALIFIER (context)

- When? Where? How? Under what conditions?
- Adds specificity

Goals when describing Needs

- ✓ Describe **WHAT** users need (not HOW to achieve it)
- ✓ Be specific and unambiguous
- ✓ Use positive framing (what to achieve, not what to avoid)
- ✓ Avoid "must" and "should" (those come later in specs)
- ✓ Be solution-neutral (doesn't prescribe technology or implementation)

Avoid these:

- ✗ Embedding solutions
- ✗ Being too vague ("easy", "better")
- ✗ Ignoring context (when/where/how used)
- ✗ Mixing multiple needs into one

More Examples

Verb	Object	Qualifier
Enable	data entry	without touching device with dirty hands
Reduce	time to record observations	compared to current manual process
Provide	confirmation	of successful data transmission
Withstand	daily use	in high-humidity greenhouse environment
Support	multiple users	without complex login procedures
Minimize	physical strain	when carrying device during 8-hour shift
Maintain	data accuracy	across temperature range of 15-35°C
Indicate	system status	through clear visual and audio cues

Quick Exercise

Refine your problem

Take your project:

1. Write 3–5 raw needs
2. Rewrite them using the correct format
3. Remove any solution words
 - Be ready to share one example

Step 2: Organizing Needs

Why organize?

You will quickly have **many needs**

- A typical product has 50-300 customer needs
- Impossible to work with flat list
- Hierarchy helps to visualize relationships and priorities
- Without structure → chaos 

Needs Hierarchy

Two levels:

Primary Needs (high-level)

- Reduce manual work
- Improve data accuracy
- Increase efficiency

Secondary Needs (more specific)

- Record data while moving in the warehouse
- Minimize typing
- Avoid errors in measurement

Example (Robotics Context)

Primary:

- Improve greenhouse monitoring

Secondary:

- Detect temperature changes quickly
- Alert users in real time
- Store data automatically

Step 3: Prioritizing Needs

Not all needs are equal

Ask:

- What matters most to users?
- What creates the most value?

Simple Prioritization Methods

Option 1: Rating (1–5)

- 5 = Critical
- 3 = Important
- 1 = Nice to have

Option 2: Ranking

- Top 3 needs only

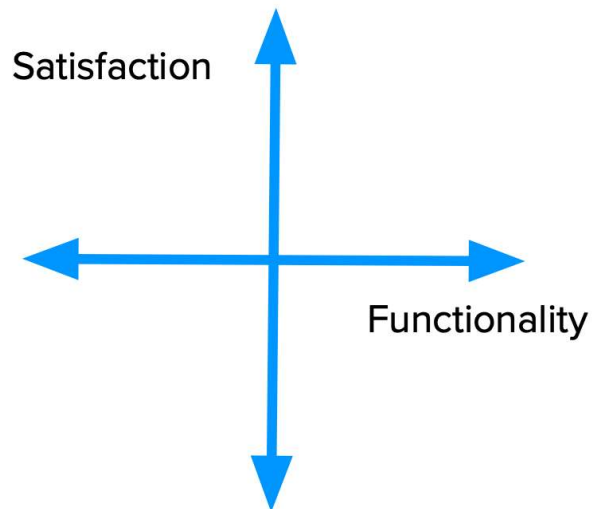
Option 3: Kano Model

The Kano Model

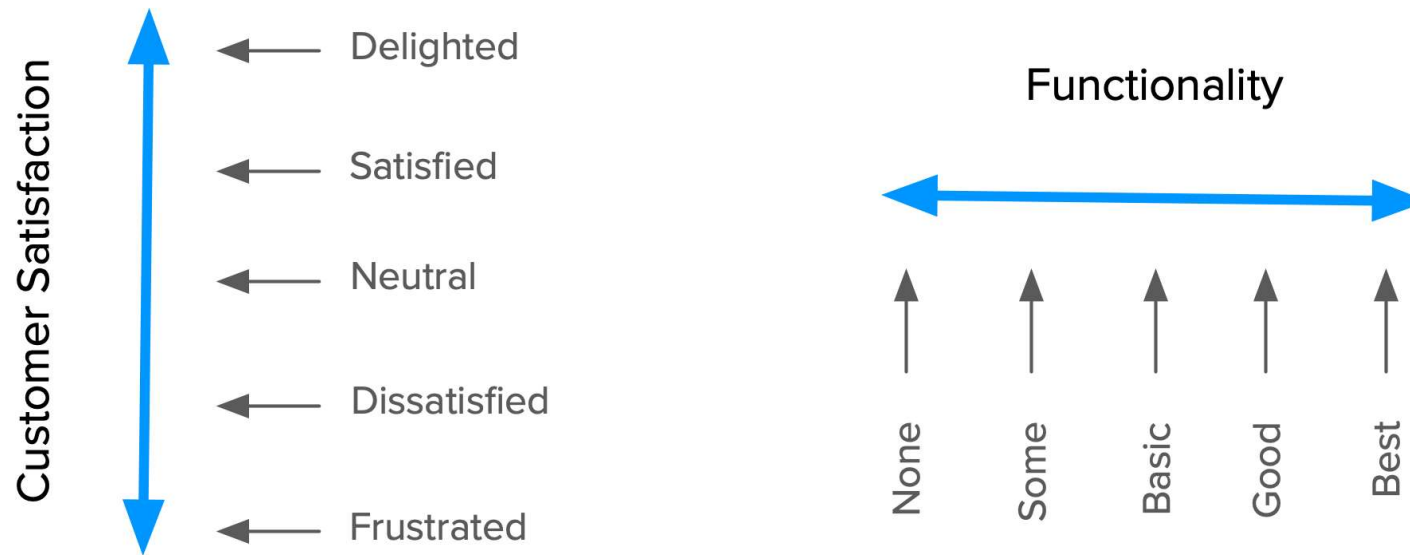
Categorizes needs by how they affect user satisfaction.

Developed by Prof. Noriaki Kano in the 1980s.

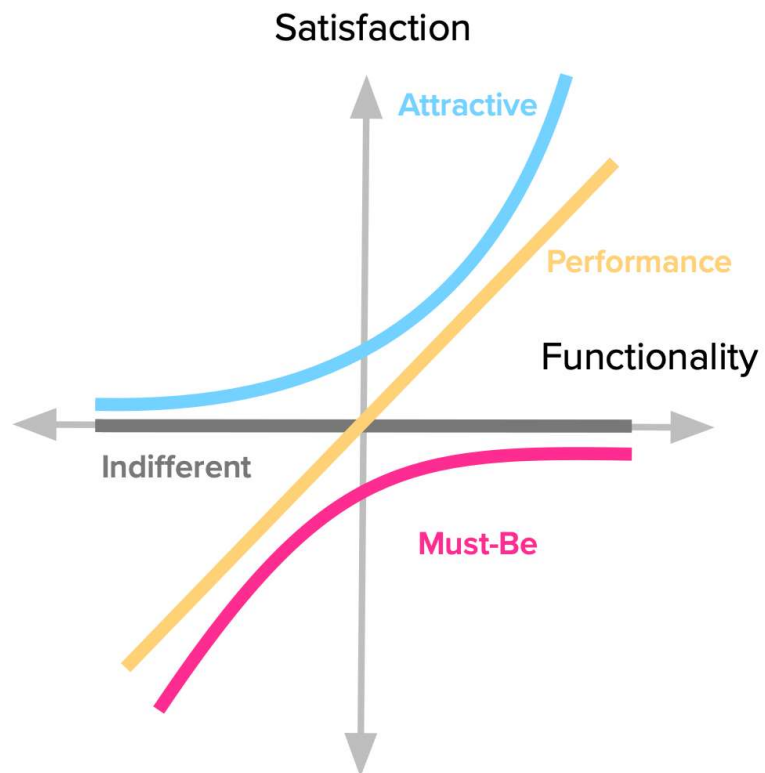
The 2 Kano Dimensions



Dimensions in the Kano Model



Categories of Needs in the Kano Model



Categories of Needs in the Kano Model (cont.)

1. **Must-Be** (Must-have, Basic)

- If missing → dissatisfaction. Example: system works reliably

2. **Performance**

- More = better. Example: speed, accuracy

3. **Attractive** (Delighters)

- Unexpected features. Example: smart suggestions

4. **Indifferent**

- You should avoid to spend resources on them, since they do not affect satisfaction

Why Kano Matters

☞ Prevents this mistake:

Unexperienced engineers/designers often design:

- Complex features ✗
- Instead of solving basic needs ✓

Step 4: From Needs → Specifications

This is a key step

Needs are:

☞ Qualitative

Specifications are:

☞ Measurable

What is a Specification?

Definition:

A specification is a measurable target

Example

Need:

Worker needs to record data quickly

Specification:

- Data entry time < 10 seconds
- Max 3 user interactions
- Error rate < 2%

Types of Specifications

- Time (seconds, minutes)
- Accuracy (%)
- Cost (€)
- Size (cm)
- Power consumption (W)
- Reliability (failure rate)

Target vs Marginal Values

Target:

- Ideal goal
- Example: < 10 seconds

Marginal:

- Minimum acceptable
- Example: < 20 seconds

Why This Matters

👉 Without specs:

- You cannot evaluate concepts
- You cannot compare solutions
- You cannot test prototypes

Workshop (for the assignment)

Your Task:

Step 1: Needs (refine)

- Write 5–10 customer needs
- Organize into hierarchy
- Assign importance

Step 2: Specifications

For each key need:

- Define 1–2 measurable specs

Example Table

Needs → Specs

Need	Importance	Metric	Target
Record data quickly	5	Time (sec)	<10 sec
Avoid errors	5	Error rate (%)	<2%
Easy to use	4	Training time	<5 min

Assignment

Customer Needs + Specifications Document

Template in Learn/Moodle

Key Takeaways

- Don't jump to solutions
- Needs $\neq$ features
- Structure creates clarity
- Specs make design testable

Thank you!

Next week: Concept Generation

Now that you have:

- ✓ Clear needs
- ✓ Measurable specs

You will generate **multiple concepts**

(No more simply random ideas)